

CSCE 713 SOFTWARE SECURITY · MODULE 02 · LECTURE NOTES

Memory Safety

The class that dominates the severe-defect data.

MODULE OUTCOMES

- Explain spatial and temporal safety violations.
- Explain each major class and its mechanism.
- Explain why the stack and heap cases differ.
- Explain what the safe languages actually guarantee.
- Find these defects in code you read.

1 Two kinds of violation

Kind	The violation	The classes
Spatial	Access outside the bounds of the intended object.	Buffer overflow and underflow, out-of-bounds read and write, off-by-one.
Temporal	Access outside the lifetime of the intended object.	Use-after-free, double free, dangling pointer to a dead stack frame.
Initialisation	Reading memory before anything was written to it.	Uninitialised memory disclosure — which leaks whatever the previous occupant left.
Type	Accessing memory as the wrong type.	Type confusion (Module 03 §4), which is adjacent and worth separating.

TEMPORAL SAFETY IS THE HARDER PROBLEM, WHICH IS WHY IT WAS SOLVED LATER

A bounds check is local. You need the object's size and the index, and both are available right at the point of access — so a compiler or a runtime can insert the check mechanically, which is exactly what every bounds-checked language does.

Lifetime is not local. Whether a pointer is still valid depends on what has happened elsewhere in the program — possibly in a different function, possibly in another thread, possibly through any of several aliases to the same object, and the information simply is not present at the point of use.

So temporal safety requires either a runtime cost — garbage collection, or reference counting — or a static discipline such as ownership and borrowing, and both are substantial design commitments that affect the whole language rather than one check.

Which is exactly why use-after-free is now the dominant class in mature C++ codebases: the spatial defects were largely found by tooling over the past two decades, and the temporal ones were not, because detecting them requires knowing a lifetime the tool cannot see (Module 09 §1 on what AddressSanitizer can and cannot catch).

2 The classes

```

BUFFER OVERFLOW (write past the end)
  overwrites whatever happens to follow: other
  variables, a saved return address, heap allocator
  metadata, a function pointer, a length field. The
  classic, and still common.

OUT-OF-BOUNDS READ
  discloses adjacent memory -- which may be another
  user's data, a cryptographic key, or a pointer
  value that defeats address randomisation
  (Module 12 section 2)

OFF-BY-ONE
  the single byte past the end. Enough to corrupt a
  length field, a null terminator, or the low byte of
  a pointer -- all of which have been exploited.

UNDERFLOW / NEGATIVE INDEX
  a signed index that went below zero, usually
  arriving from an integer defect (Module 03)

AND THE COMMON CAUSES
  a length taken from the input and trusted
  a size computed with an arithmetic overflow
  a string function with no bound argument
  and a check that used the WRONG buffer's size

```

'A check that used the wrong buffer's size' is badly underrated. It survives code review because the check is visibly present — the reviewer sees a bounds comparison, ticks it, and moves on — and it

survives testing because the two buffers are usually the same size in the common case. **Which makes it a defect that only careful reading or a sanitiser finds**, and it is worth looking for specifically whenever two buffers appear in one function.

USE-AFTER-FREE

the allocator may have reissued that memory to somebody else. An attacker who controls an allocation between the free and the use controls what the stale pointer now points at -- which turns a dangling read into a controlled read.

DOUBLE FREE

corrupts allocator metadata, which is itself a write primitive in many allocator designs.

DANGLING POINTER TO THE STACK

returning a pointer to a local, or retaining one after the frame is gone. The memory is reused by the very next call.

AND THE PATTERNS THAT PRODUCE THEM

an error path that frees and then continues
two owners with unclear responsibility
a cached pointer that outlives its object
a container reallocating while a pointer into it is
still held -- iterator invalidation
and a callback that fires after teardown

Iterator invalidation is the one that looks like entirely ordinary code. A push_back during iteration is a use-after-free with no free anywhere in sight — the container reallocated and the held pointer or iterator now references the old buffer. **Which makes the class feel considerably less exotic**, and is the example worth leading with when explaining it to people who believe use-after-free is an exotic C problem.

3 Stack and heap

- **The stack holds return addresses and saved registers**, so an overflow there historically gave direct control of execution by overwriting where the function would return to — **which is precisely why stack canaries exist** (Module 12 §2).
- **The stack is also highly predictable in layout**, which is what made the classic attack reliable — and which address space randomisation partly addresses by making the addresses unknown rather than the layout different.
- **The heap holds allocator metadata and other live objects** — so a heap overflow corrupts a neighbour, and **the attacker's real work is in arranging which neighbour it is**, by causing allocations and frees in a chosen order.
- **Which is why heap exploitation is fundamentally about allocation control rather than about the overflow itself** — and why allocator hardening (randomised placement, metadata separation, guard pages, quarantining freed memory) is a genuine mitigation rather than a cosmetic one.
- **And both regions are now substantially mitigated rather than fixed**, which Module 12 treats honestly. **The shift from predominantly stack defects to predominantly heap defects over two decades was driven by the mitigations** — the stack was hardened first, so the attention moved, which is a useful illustration of CSCE 701 Module 01's asymmetry operating over a long timescale.

4 What safe languages actually give you

A MEMORY-SAFE LANGUAGE ELIMINATES THE CLASS RATHER THAN REDUCING THE DEFECT RATE

Rust, Go, Java, Python, and C# do not have exploitable buffer overflows or use-after-free in safe code — not fewer of them, but structurally none, because the error is not expressible in the language. That is a categorical difference from a lower defect rate, and it is the whole of the argument.

And the evidence is strong. Large projects with long-running security programmes report that memory safety accounted for the clear majority of their severe vulnerabilities, and that newly written code in a safe language contributes essentially none of them — a result that has now been reported independently by several organisations with very different codebases.

But the gaps are real and worth naming: unsafe blocks, foreign function interfaces to C libraries, the language runtime itself, and logic defects which are entirely unaffected — an authorisation check you forgot is just as missing in Rust (CSCE 701 Module 03).

So the honest claim is narrow and still very large: 'eliminates this class of defect in safe code' — which is most of the severe-defect data and is **not all of security**, and stating it that way is both more defensible and more persuasive than the broader version (Module 13).

- **Run the sanitisers in continuous integration** (Module 09) — AddressSanitizer finds most spatial defects and many temporal ones, and it is **the single highest-value change available to an existing C or C++ codebase**, requiring no source changes at all.

- **Fuzz the parsers** (Module 08), because that is precisely where attacker-controlled input meets manual memory handling — and it is where the published defect data concentrates.
- **Use the containers and the smart pointers, and treat raw pointer arithmetic as a review trigger** rather than as ordinary code — which is a cheap cultural change with measurable effect.
- **Enable the compiler's hardening** — the bounds-checked standard library modes, the fortified string functions, and the relevant warnings promoted to errors (Module 12 §2).
- **And write new components in a safe language wherever the interface boundary permits it**, which is the incremental version of the argument above and does not require anybody to approve a rewrite. **Sanitisers in CI plus fuzzing the parsers is the highest-yield pair**, and **both are achievable without rewriting anything**, which is why they are the recommendation rather than the language change.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Dowd, McDonald & Schuh, chapters 5 through 7	§2 and §3 in depth — the mechanisms explained at the level of what the machine does. Library copy.
Szekeres et al. — SoK: Eternal War in Memory (free)	§1's taxonomy and §3's mitigation history , systematised — the single best overview of this material.
MIT 6.858's buffer overflow lab (free, OCW)	§2 and §3 practically , on targets built for the purpose — which is Module 01 §4's constraint satisfied.
Google and Microsoft's memory safety reports (free)	§4's evidence — measured proportions of severe defects attributable to memory safety, from codebases at scale.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Classify ten CVEs** as spatial, temporal, initialisation, or type.
- 2 **Explain why a bounds check is local** and a lifetime check is not.
- 3 **Write a minimal off-by-one** and observe what the extra byte corrupts.
- 4 **Write a wrong-buffer-size check** and confirm review would miss it.
- 5 **Trigger an iterator invalidation** and catch it with AddressSanitizer.
- 6 **Write a use-after-free** and observe what the memory contains after reallocation.
- 7 **Compare stack and heap overflow consequences** in two minimal programs.

- 8 **Enable AddressSanitizer on an existing project** and report what it finds.
- 9 **State the safe-language claim precisely**, including the four gaps.
- 10 **Find one logic defect** that a safe language would not have prevented.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name the two kinds of violation and the classes under each.
- 2 Why is temporal safety harder, and what are the two solutions?
- 3 Why does use-after-free now dominate in mature C++ codebases?
- 4 Name four spatial classes and four common causes.
- 5 Why is the wrong-buffer-size check underrated?
- 6 Name four patterns that produce temporal defects.
- 7 Why does iterator invalidation look like ordinary code?
- 8 Why is heap exploitation about allocation control?
- 9 State precisely what a safe language gives, and the four gaps.
- 10 What is the highest-yield pair for existing C and C++?